

DELPH-IN Unified Grammar Configuration File Format

When loading a grammar, need to specify which files to load, how to load them, and processing parameters

Loading currently by `script` (LKB) or `config.tdl` (ACE / PyDelphin ...)

Want to converge on a single mechanism in order to: avoid duplication of information, abstract over unimportant differences between the various grammar processors, reduce mental burden on grammar developers

Unified configuration file format under consideration since 2012, primarily driven by Michael Goodman

- SIG in 2020: <https://github.com/delph-in/docs/wiki/VirtualSharedConfigs>
- proposal: <https://github.com/delph-in/docs/wiki/GrammarConfigurationRfc>
- based on TDL: <https://github.com/delph-in/docs/wiki/TdIRfc>

Experimental Implementation in LKB-FOS

Based on RFCs

- `:begin / :end :type / :instance` and `:include` as in TdlRFC
- `:begin / :end :config` as in GrammarConfigurationRfc, optionally condition-alised following Sub-Proposal 1 “Another possible syntax...”
- config environment processor identifiers are looked up in Lisp built-in global variable `*features*` (containing LKB, LKB-FOS, ...)
- config environments may contain parameter settings, looking a bit like type definitions except RHS is a non-empty sequence of identifiers and/or strings

Extends TdlRFC and ACE implementation

- further `:instance` status values `label` and `root`
- further parameters beyond those in AceConfiguration, including: `load-lkb-preferences`, and `preload- / postload-lisp-files`

Tested on ERG 2025, based on `ace/config.tdl` and `english.tdl`, merging into them `lkb/script` and parts of `lkb/globals.lsp`

Examples for discussion: `lkb/config.tdl`, `english-lkb.tdl`

!!! – issues to note or resolve

*** – possibly unintentional divergences

Several practical issues

- LKB must finalise type hierarchy before processing any other constructs; possible approaches
 - accept only `:include` statements inside type environment, or
 - read and save definitions until they can be executed in some safe order
- similar restriction on lexical entry environment
- are conditionals too verbose? i.e., `:begin :config processor. ... :end :config.`
- between LKB and ACE some important parameters have exact counterparts, but many more do not – especially those to do with semantics
- no standard notation for parameter values that are paths or empty lists
- do repeated parameter settings cause conflict or reassignment?
- issues marked with `!!!` in examples